



general crypto charity fiction art music science logifusion upetahelp 🔍



Satchmo earned \$2.62

#crypto

MetaNet Permissioning - Granting Users Exclusive Ownership of a Sub-Tree

👍 VOTE 25¢ \$3.00 ⌚ 1yr ago 💰 50.0¢ 💬 1

Another day another fundamental shift in my understanding of how something in Bitcoin works. Today, it was permissioning properties of the Metanet protocol. I always assumed that since the signature of a parent node is required to create a Metanet edge, it is safe to assume all child nodes inherit the same permissions and cannot disavow their authority. As it turns out, this is not the case. This excellent article from Jack Davies comes very close to this topic but does not specifically mention the very simple technique I will describe that unlocks this property.

Edge Cases: The Metanet Blog

Part 2.1 — Nodes & Edges

Medium / 2019年9月2日

<https://medium.com/nchain/edge-cases-the-metanet-blog-c0e794d1c9c5>

In this article, Jack explains that the following Metanet edge between Alice and Bob is typically not possible since Bob has no ability to sign an input from Alice's tree:

What I've discovered is, a slight change to the Node B creation process can in fact make this edge possible, and it has some unique properties.

Multi-User Permission w/ Inheritance

When considering what types of metanet structures might be useful for Bitcoin applications, it was always clear that Alice could simply share a child node's private key with Bob, thereby giving him access to write to a sub-tree derived from the same extended keypair as the root node. Doing this would of course allow Alice to write to the sub-tree as well. Bob now has permission to Node B and below, but not exclusively. Alice retains permission as well. This image illustrates an example, where the CEO has full access to all descendant nodes, and descendants have no write access to their ancestor nodes.

<https://bico.media/9739d91f611135b12674c6f92236d1b52074c82e572e24121cc4b15db64b50b5>

Side note: This process must be done with care. If metanet node keys meet these conditions, sharing a single child private key can allow Bob to derive the keys for the parent node as well as all siblings:

- 1) Addresses are derived using the traditional HD wallet scheme*
- 2) Addresses are non-hardened*
- 3) Extended public key is published/exposed*

To protect against this you could create a "firewall" node using a hardened key in between the graph and the portion to be shared with a

second party to prevent exposing other keys in the graph above that node.

Mutually Exclusive Multi-User Permission

I really wanted the user to keep full control of their contributions to the graph. Up until now, I had imagined Bob would create his own graph with his own keys. This graph could be linked to an Alice's graph using something like Author Identity Protocol. This would work, but because AIP is not linked to tx inputs, but it requires supplemental verification to be effective. In addition, the tools we use to look at a particular metanet graph would have to be modified to use this secondary protocol to associate the two trees. A query for the root node of Alice's tree would not produce Bob's tree in the results.

Today though, I realized something very simple, but extremely powerful that I have not heard mentioned before. You can actually give Bob **exclusive** access to his tree without any of the security concerns mentioned above, or logistical complications of using multiple protocols. It is possible to create a sub-tree where Alice has no permission to extend the Bob's tree, even though it is under her tree as well. Alice cannot derive child keypairs beyond Bob's root node. It is the equivalent of granting exclusive ownership to a section of a metanet tree.

It only requires that you break from the "HD wallet" convention where each child node address is derived from the same set of HD keys and think a bit outside the box:

The Solution

Assume:

- **Alice** is the root node key holder.
- **Bob** is some second party who will receive exclusive permissions to a portion of party a's graph.

Step 1:

Alice simply asks **Bob** for an address. This can be the first address of a brand new extended keypair for example. This address is fully under Bob's control and he never shares the private key with Alice.

Step 2:

Alice signs from an input (Sig Pa), referencing **Bob's** address as Pb and signs the input as per metanet protocol specifications.

<https://bico.media/00844c20e91792ac4ff87428cff4c45e4047fd8e6c2b1b3d49d4473c5264213c>

Practical Use-Case

This may be useful if you have a graph representing a Bitcoin application, and you want to give a user exclusive rights to their 'account', while keeping

everything within the application's graph.

Taking this idea a bit further, if the app's metanet wallet exists on the server side where the user has no access to the application's keys, and a user's metanet wallet exists on the client side where the application owner has no access, this can be made an automatic process.

The front-end software can generate a new HD wallet for the user, and share the first address with the application server. The server can then create a new metanet node referencing that address, effectively handing over control to the user. The user's address will act as a root node of the new sub-tree, and it will be read from metanet APIs such as Unwriter's MOM as a single graph. The user only needs to back up a key once their "sub-tree" root node is created. Pretty neat.

Implications

There is one important thing to note here. Without using a standard HD key scheme and exposing the extended public key of the top level node, and then validating the address of each node matches its position in the tree, it is not possible to know where exactly the ownership change occurs. One example of where this may become relevant is, we cannot assume that an attestation applied to a root node automatically propagates all the way down a graph. Keys are very much not identity, and even when keys are explicitly related, identity will not necessarily flow across all metanet edges.

Multi-Player Metanet

Another really cool use-case for this is the ability to hand control back and forth between users. Props to @Deggen for pointing this out. We can make that flow

pretty nice:

- Alice starts the game by immediately writing a nLockTime tx which will hand over write permission to Bob in x minutes.
- If Alice completes her turn before the timer expires, she writes a node representing her move, and gives control to Bob. This tx spends the same exact utxo as the original nLockTime tx, invalidating it.
- Bob repeats this process, immediately writing an nLockTime tx representing the turn timer, and we go back and forth like this until the match is complete.

How awesome is that!?

 **BUY 50.0¢**

Join Yours to support Satchmo

Yours allows readers to support their favorite content creators directly. Users can also earn money by selling their content, voting on good content and more.

[or login](#)

[Join](#)

 **VOTE 25¢**

\$3.00

 **TIP 25.0¢** ▼

\$1.12

report share



No one has reviewed this piece of content yet

What a great post - excellent work!

 **COMMENT 10.0¢**

Comments

 **John Pitts** spent 2.0¢

Want to understand this, but too much Alice & Bob. Was digging on CEO example but then it went quickly back to Alice/Bob. Whose node? Alice and Bob node owners, users? Too many pronouns. Hats off to others for understanding it. I need it to be in regular human, ha!

 **TIP 25.0¢** ▼

0.0¢

12mo ago

 2.0¢

[Twitter](#) [Facebook](#) [YouTube](#) [FAQ](#) [About](#)

Have a problem? Please contact support@yours.org.